

**Автономная некоммерческая организация высшего образования
«Университет Иннополис»**

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 «Информатика и вычислительная техника»**

**FINAL QUALIFICATION WORK
(BACHELOR'S THESIS)
Academic Program
09.03.01 «Computer Science»**

**Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»**

**Field of Study:
«Computer Science»**

Тема

Исследование Kubernetes как платформы для бизнес-логики с использованием Custom Resource Definitions и паттерна Operator

Topic

Exploring Kubernetes as a Platform for Business Logic Using Custom Resource Definitions and the Operator Pattern

Работу выполнил /
Prepared by

**Бобкунов Евгений /
Evgeny Bobkunov**

подпись / signature


Руководитель
выпускной
квалификационной
работы / *Final
Qualification Work
Supervisor*

**Крид Дмитрий /
Dmitrii Creed**

подпись / signature

Консультанты
Consultants

подпись / signature

Иннополис, Innopolis, 2026

Contents

1	Introduction	12
1.1	Background and Context	12
1.2	Problem Statement	13
1.3	Research Questions	14
1.4	Research Objectives and Contributions	15
1.5	Research Methodology	16
1.6	Thesis Organization	17
1.7	Summary	18
2	Literature Review	19
2.1	Introduction	19
2.2	Kubernetes Architecture and Core Concepts	19
2.2.1	Container Orchestration and Kubernetes	19
2.2.2	Declarative API Model	20
2.2.3	Controller Pattern and Reconciliation Loop	21
2.3	Custom Resource Definitions	21
2.3.1	Extending the Kubernetes API	21
2.3.2	CRD Features and Capabilities	22
2.3.3	Performance and Scalability Considerations	22

2.4	The Operator Pattern	22
2.4.1	Operator Fundamentals	22
2.4.2	Operator Development Frameworks	23
2.4.3	Operator Use Cases and Applications	23
2.5	Related Systems and Approaches	24
2.5.1	Infrastructure as Code and GitOps	24
2.5.2	Service Mesh Architectures	24
2.6	Observability in Cloud-Native Systems	25
2.6.1	Monitoring and Metrics Collection	25
2.6.2	Distributed Tracing	25
2.7	Performance Studies and Challenges	26
2.7.1	Kubernetes Performance Characteristics	26
2.7.2	Complexity and Operational Considerations	26
2.8	Research Gaps	27
2.9	Summary	27
3	Methodology	29
3.1	Research Approach	29
3.2	Business Domain Modeling	30
3.2.1	Core Entities	30
3.2.2	Business Logic	30
3.3	Traditional System Architecture	31
3.3.1	Technology Stack	31
3.3.2	Architecture Overview	32
3.3.3	Execution Model	32
3.3.4	Data Persistence	33
3.3.5	API Endpoints	34

3.4	Kubernetes-Based System Architecture	34
3.4.1	Technology Stack	34
3.4.2	Custom Resource Definitions	35
3.4.3	Operator Controller Implementation	37
3.4.4	Execution Model	39
3.4.5	Data Persistence	40
3.4.6	Observability Integration	40
3.4.7	Deployment Configuration	42
3.5	Comparison of Architectural Approaches	42
3.5.1	State Management	43
3.5.2	Execution Flow	44
3.5.3	Time-Based Logic	44
3.5.4	Lifecycle Management	44
3.6	Observability Design	45
3.6.1	Kubernetes System Observability	45
3.6.2	Traditional System Observability	47
3.6.3	Comparative Analysis	47
3.7	Experimental Methodology	48
3.7.1	Evaluation Environment	48
3.7.2	Evaluation Metrics	48
3.7.3	Benchmark Methodology	50
3.7.4	Threats to Validity	51
3.8	Summary	51
4	Implementation	52
4.1	Implementation Overview	52
4.2	Kubernetes-Based System (kube-billing)	53

4.2.1	Project Structure	53
4.2.2	CRD Implementation	54
4.2.3	Controller Implementation	56
4.2.4	Metrics Implementation	59
4.2.5	Tracing Implementation	60
4.2.6	Deployment Configuration	61
4.3	Traditional System (backend-billing)	62
4.3.1	Project Structure	62
4.3.2	Data Models	63
4.3.3	HTTP Handlers	64
4.3.4	Repository Layer	65
4.3.5	Background Worker	65
4.3.6	Validation	66
4.3.7	Observability	67
4.4	Implementation Comparison	68
4.4.1	Code Metrics	68
4.4.2	State Management	69
4.4.3	Error Handling	70
4.4.4	Testing Approach	71
4.5	Implementation Challenges	71
4.5.1	Kubernetes System Challenges	71
4.5.2	Traditional System Challenges	71
4.6	Summary	72
5	Evaluation and Discussion	73
5.1	Experimental Setup	73
5.1.1	Test Environment	73

5.1.2	Benchmark Tools	74
5.2	Code Metrics Comparison	75
5.2.1	Lines of Code and File Count	75
5.2.2	Cyclomatic Complexity	75
5.2.3	Dependencies	76
5.2.4	Test Coverage	76
5.3	Performance Evaluation	77
5.3.1	Microbenchmarks	77
5.3.2	End-to-End Latency	77
5.3.3	Throughput Under Load	78
5.4	Resource Utilization	78
5.4.1	CPU Usage	78
5.4.2	Memory Usage	79
5.4.3	Startup Time	79
5.5	Operational Characteristics	80
5.5.1	Scaling	80
5.5.2	Fault Tolerance	81
5.5.3	Observability Coverage	81
5.6	Summary of Findings	82
5.7	Discussion	82
5.7.1	When to Use Kubernetes Operator Approach	82
5.7.2	When to Use Traditional Backend Approach	83
5.7.3	Threats to Validity	83
5.8	Summary	84
6	Conclusion	85
6.1	Summary of Research	85

6.2	Answers to Research Questions	86
6.2.1	RQ1: How can business logic be effectively modeled and implemented using Kubernetes CRDs and the Operator pattern?	86
6.2.2	RQ2: What are the performance characteristics of CRD/Operator-based implementation compared to traditional REST API architectures?	87
6.2.3	RQ3: What are the operational implications and observability capabilities of using Kubernetes as a business logic platform?	87
6.2.4	RQ4: Under what circumstances is the CRD/Operator approach advantageous, and what are its limitations?	88
6.3	Contributions	89
6.4	Limitations	90
6.5	Future Work	91
6.5.1	Technical Improvements	91
6.5.2	Research Extensions	92
6.5.3	Production Deployment	92
6.6	Final Remarks	93
6.7	Publications and Artifacts	93
	Bibliography cited	95

Abstract

This thesis explores the applicability of Kubernetes as a platform for implementing business logic using Custom Resource Definitions (CRDs) and the Operator pattern. While Kubernetes has become the de facto standard for container orchestration, its potential as a business logic platform remains understudied.

Two functionally equivalent billing systems were designed and implemented: (1) a traditional Go backend with REST API and PostgreSQL database, and (2) a Kubernetes Operator with CRDs and reconcile-based control loop. Both systems implement identical business logic (billing plan management, subscription lifecycle, recurring billing, expiration handling) but differ fundamentally in architecture, state management, and execution model.

Comprehensive evaluation was conducted across multiple dimensions: code metrics, performance benchmarks, resource utilization, and operational characteristics. The results demonstrate that the traditional backend achieves 10–20 $\times$ lower latency (p50: 10ms vs 100ms for subscription creation) and 5 $\times$ higher throughput (1,000 req/s vs 200 req/s). However, the Kubernetes Operator provides superior operational automation: horizontal pod autoscaling, self-healing, built-in observability (Kubernetes Events, Conditions), and GitOps compatibility.

The research contributes empirical evidence to the cloud-native architecture discourse, moving beyond anecdotal claims to data-driven decision making. A decision framework is provided for practitioners: the traditional approach is recommended for performance-critical, portable applications with simple deployment requirements; the Operator approach is suitable for organizations already

invested in Kubernetes ecosystem that prioritize operational automation over raw performance.

Both implementations are released as open-source, serving as reference architectures for similar projects. The findings suggest that while Kubernetes can effectively serve as a business logic platform, the choice between approaches should be guided by organizational context, team expertise, and specific business requirements rather than technological trends alone.

Abstract

[russian]

Данная диссертация исследует применимость Kubernetes в качестве платформы для реализации бизнес-логики с использованием Custom Resource Definitions (CRD) и паттерна Operator. Несмотря на то, что Kubernetes стал стандартом де-факто для оркестрации контейнеров, его потенциал как платформы для бизнес-логики остаётся недостаточно изученным.

Были спроектированы и реализованы две функционально эквивалентные биллинг-системы: (1) традиционный Go-бэкенд с REST API и базой данных PostgreSQL, (2) Kubernetes Operator с CRD и циклом reconcile. Обе системы реализуют идентичную бизнес-логику (управление тарифными планами, жизненный цикл подписок, периодический биллинг, обработка истечения срока), но фундаментально различаются архитектурой, управлением состоянием и моделью выполнения.

Проведено комплексное сравнение по нескольким измерениям: метрики кода, бенчмарки производительности, использование ресурсов и операционные характеристики. Результаты показывают, что традиционный бэкенд обеспечивает в 10–20 раз меньшую задержку (p50: 10 мс против 100 мс для создания подписки) и в 5 раз более высокую пропускную способность (1000 запросов/с против 200 запросов/с). Однако Kubernetes Operator предоставляет превосходную операционную автоматизацию: автоматическое масштабирование (HPA), самовосстановление, встроенную наблюдаемость (Kubernetes Events, Conditions) и совместимость с GitOps.

Исследование вносит эмпирические данные в дискуссию об облачных нативных архитектурах, переходя от анекдотических утверждений к обоснованным решениям. Предоставлена рамка принятия решений для практиков: традиционный подход рекомендуется для критичных к производительности приложений с простыми требованиями к развёртыванию; подход с Operator подходит для организаций, уже инвестирующих в экосистему Kubernetes и приоритизирующих операционную автоматизацию над производительностью.

Обе реализации выпущены как open-source, служа референсными архитектурами для аналогичных проектов. Выводы показывают, что, хотя Kubernetes может эффективно служить платформой для бизнес-логики, выбор между подходами должен определяться организационным контекстом, экспертизой команды и конкретными бизнес-требованиями, а не только технологическими трендами.

Chapter 1

Introduction

1.1 Background and Context

The rapid evolution of cloud-native computing has fundamentally transformed how modern software systems are architected, deployed, and operated. At the core of this transformation lies Kubernetes, which has emerged as the de facto standard for container orchestration since its open-source release in 2014 [1], [2]. Initially developed by Google based on their internal Borg system and later donated to the Cloud Native Computing Foundation (CNCF), Kubernetes provides robust capabilities for automating deployment, scaling, and management of containerized applications [3], [4].

However, Kubernetes offers far more than container orchestration capabilities. Its extensible architecture, built around a declarative API model and a control plane pattern, enables it to function as a universal control plane for managing arbitrary resources [5]. Through Custom Resource Definitions (CRDs) and the Operator pattern, Kubernetes can be extended to manage not only infrastructure components but potentially business logic itself [6]–[8]. This extensibility has been

thoroughly documented in both academic literature and industry best practices [9], [10].

Modern cloud systems strive for maximum automation and flexibility. Organizations are increasingly seeking unified platforms that can manage both infrastructure and application logic through consistent interfaces [11]. The declarative nature of Kubernetes, combined with its powerful extension mechanisms, positions it as a potential candidate for implementing business logic, not just orchestrating containers. This paradigm shift raises fundamental questions about the boundaries of the Infrastructure-as-Code approach and whether Kubernetes can serve as a unified control plane for both infrastructure and application concerns [12].

1.2 Problem Statement

Despite the widespread adoption of Kubernetes for infrastructure management, there remains a lack of systematic study regarding its applicability for implementing business logic using CRDs and the Operator pattern. Traditional application architectures typically separate business logic implementation from infrastructure concerns, with applications deployed as microservices that implement domain-specific functionality through conventional programming approaches.

Several projects have demonstrated the power of extending Kubernetes through CRDs and Operators, including Crossplane for infrastructure provisioning [13], Kubeflow for machine learning workflows, and various database operators [14]. However, most existing solutions focus on infrastructure management or specialized workloads rather than general-purpose business logic [15].

The main problem lies in understanding whether and how Kubernetes, through CRDs and Operators, can effectively serve as a platform for implementing

business logic. Key questions include: What are the performance characteristics of such an approach? How does it compare to traditional REST API-based architectures? What are the operational implications? What types of business problems are well-suited to this paradigm?

Solving this problem is important for several reasons. First, it could enable organizations to consolidate their technology stacks by using Kubernetes as a unified platform. Second, it may provide new patterns for building highly automated, self-healing business applications. Third, it could help determine clear boundaries for when this approach is beneficial versus when traditional architectures should be preferred.

1.3 Research Questions

This thesis investigates the following research questions:

RQ1: How can business logic be effectively modeled and implemented using Kubernetes Custom Resource Definitions and the Operator pattern?

RQ2: What are the performance characteristics of a business logic implementation based on CRDs and Operators compared to traditional REST API architectures?

RQ3: What are the operational implications and observability capabilities of using Kubernetes as a business logic platform?

RQ4: Under what circumstances is the CRD/Operator approach advantageous for implementing business logic, and what are its limitations?

1.4 Research Objectives and Contributions

The primary objective of this research is to explore and evaluate Kubernetes as a platform for implementing business logic through Custom Resource Definitions and the Operator pattern. Specific objectives include:

1. Study existing approaches to extending the Kubernetes API through CRDs and Operators, analyzing patterns and best practices established by the cloud-native community [9], [10].
2. Design and develop a prototype application that implements representative business logic using CRDs and Operators, demonstrating the feasibility of this approach.
3. Establish comprehensive observability for the prototype system using cloud-native tools including Prometheus and OpenTelemetry to enable detailed performance analysis [16]–[18].
4. Conduct experimental comparisons between the CRD/Operator-based implementation and a traditional architecture using PostgreSQL with REST API.
5. Analyze results across multiple dimensions including reaction time to events, implementation complexity, observability capabilities, and system reliability.

The expected contributions of this work include:

- A systematic analysis of using Kubernetes CRDs and Operators for business logic implementation, filling a gap in current research [11].

- A working prototype demonstrating practical patterns for modeling business entities and processes as Kubernetes custom resources.
- Quantitative performance comparisons providing empirical evidence of the strengths and weaknesses of this approach.
- Practical recommendations and guidelines for practitioners considering this architectural approach, including identification of suitable use cases and potential pitfalls.
- Insights into the boundaries of Kubernetes extensibility and the Infrastructure-as-Code paradigm.

1.5 Research Methodology

This research employs a mixed-methods approach combining system design, implementation, and experimental evaluation:

Phase 1 — Literature Review and Architecture Design: Conduct comprehensive review of existing literature on Kubernetes extensibility, CRDs, Operators, and related patterns [2], [8], [9]. Design the architecture for both the CRD/Operator-based system and the baseline traditional system.

Phase 2 — Implementation: Develop the prototype systems using appropriate tools and frameworks. The CRD/Operator implementation will utilize Kubebuilder for scaffolding and controller development [15]. Both systems will implement equivalent business logic to enable fair comparison.

Phase 3 — Observability Setup: Configure comprehensive monitoring using Prometheus for metrics collection and OpenTelemetry for distributed trac-

ing [16]–[18]. Implement GitOps practices using ArgoCD for deployment management [19], [20].

Phase 4 — Experimental Evaluation: Conduct controlled experiments measuring key performance indicators including reaction time to events, implementation complexity, observability coverage, and reliability [21].

Phase 5 — Analysis and Conclusions: Analyze experimental results, synthesize findings, and formulate conclusions on the applicability of the approach.

1.6 Thesis Organization

The remainder of this thesis is organized as follows:

Chapter 2 — Literature Review: Provides comprehensive background on Kubernetes architecture, Custom Resource Definitions, the Operator pattern, and related work in extending Kubernetes for business logic. Reviews relevant observability tools and GitOps practices.

Chapter 3 — Methodology: Describes the design of both the CRD/Operator-based system and the traditional baseline system. Details the business logic domain selected for implementation and architectural decisions.

Chapter 4 — Implementation: Covers implementation details including CRD specifications, Operator controller logic, observability instrumentation, and deployment configuration.

Chapter 5 — Experimental Evaluation: Presents the experimental setup, methodology, and results of performance comparisons between the two approaches.

Chapter 6 — Discussion: Analyzes results, discusses implications, identifies suitable use cases, and provides recommendations for practitioners.

Chapter 7 — Conclusion: Summarizes key findings, discusses limitations,

and suggests directions for future work.

1.7 Summary

This chapter has introduced the research problem of using Kubernetes as a platform for business logic through Custom Resource Definitions and the Operator pattern. While Kubernetes has proven highly successful for infrastructure management, its applicability for implementing business logic remains understudied [11]. This thesis aims to address this gap through systematic design, implementation, and evaluation of both CRD/Operator-based and traditional architectures. The expected outcomes include practical insights into when and how this approach can be beneficial, contributing to the broader understanding of cloud-native application development patterns.

Chapter 2

Literature Review

2.1 Introduction

This chapter provides a comprehensive review of the literature and technologies relevant to exploring Kubernetes as a platform for business logic implementation. The chapter is organized into several sections covering Kubernetes architecture and extensibility, Custom Resource Definitions and Operators, related approaches to extending Kubernetes, observability in cloud-native systems, and performance considerations.

2.2 Kubernetes Architecture and Core Concepts

2.2.1 Container Orchestration and Kubernetes

Kubernetes, initially developed by Google and open-sourced in 2014, has become the leading platform for container orchestration [1], [2]. The system builds upon years of experience running production workloads at Google, combined with best-of-breed ideas from the open-source community [3], [4]. Kubernetes

automates deployment, scaling, and management of containerized applications, providing a platform-agnostic abstraction layer over underlying infrastructure.

The architecture of Kubernetes follows a control plane pattern, where a centralized control plane manages the desired state of the cluster, and worker nodes execute containerized workloads [2], [22]. The control plane consists of several key components: the API server, which exposes the Kubernetes API and serves as the entry point for all operations; etcd, a distributed key-value store that maintains cluster state; the scheduler, which assigns pods to nodes; and the controller manager, which runs controllers to reconcile actual state with desired state [10].

2.2.2 Declarative API Model

A distinguishing feature of Kubernetes is its declarative API model [5]. Rather than imperatively instructing the system on how to perform operations, users declare the desired state of resources, and Kubernetes automatically reconciles the actual state to match. This declarative approach enables powerful automation capabilities and forms the foundation for extending Kubernetes functionality [9].

The Kubernetes API is organized into API groups and versions, providing a structured approach to API evolution and extensibility [23]. Core resources like Pods, Services, and Deployments are part of the built-in API, while custom resources can be added through extension mechanisms. This design has been validated through extensive production use and is thoroughly documented in standard references [2], [4].

2.2.3 Controller Pattern and Reconciliation Loop

Kubernetes extensively uses the controller pattern for managing resources [10], [24]. A controller watches for changes to resources of interest and takes actions to reconcile the actual state with the desired state specified in the resource definition. This reconciliation loop runs continuously, enabling self-healing and automated management capabilities [8].

Controllers operate independently and asynchronously, communicating only through the Kubernetes API server. This architectural pattern enables high scalability and resilience, as controllers can be distributed across multiple instances and automatically recover from failures [9]. The controller pattern has become a fundamental building block for cloud-native applications and is extensively covered in Kubernetes design patterns literature [10].

2.3 Custom Resource Definitions

2.3.1 Extending the Kubernetes API

Kubernetes provides mechanisms for extending its API, with Custom Resource Definitions (CRDs) being the primary approach [9], [25]. CRDs represent a declarative approach to defining new resource types without requiring custom API server code, making them accessible to operators and developers [2].

A CRD defines a new resource type, including its API group, version, kind, and schema, making it a first-class citizen in the Kubernetes API [6], [9]. This approach enables extending Kubernetes to manage domain-specific resources while maintaining consistency with the platform's declarative model.

2.3.2 CRD Features and Capabilities

Modern CRDs support advanced features including schema validation using OpenAPI v3, multiple versions with conversion webhooks, status subresources for privilege separation, custom columns in kubectl output, and structural schemas for efficient storage and processing [9], [26]. These capabilities enable CRDs to provide rich, production-ready APIs comparable to built-in Kubernetes resources.

CRDs enable developers to model custom workflows, advanced configurations, and support for domain-specific requirements that go beyond Kubernetes' built-in resources. The flexibility of CRDs has made them a cornerstone of cloud-native extensibility, with hundreds of widely-used CRDs available in the ecosystem [11].

2.3.3 Performance and Scalability Considerations

The performance characteristics of CRDs depend heavily on the underlying etcd datastore, which stores all cluster state. Research on etcd performance shows that at scale, write latency can become a bottleneck, particularly for clusters with many CRD instances [27]. Organizations deploying CRD-intensive applications must carefully consider etcd capacity planning and implement appropriate monitoring.

2.4 The Operator Pattern

2.4.1 Operator Fundamentals

The Operator pattern represents a methodology for packaging, deploying, and managing Kubernetes applications by extending the Kubernetes API [7], [8].

Operators combine CRDs with custom controllers to encode operational knowledge about managing complex applications, effectively acting as automated site reliability engineers [14].

Operators capture the aim of human operators who manage services—they have deep knowledge of how systems should behave, how to deploy them, and how to react to problems. By codifying this operational knowledge, Operators enable applications to be self-managing, automatically handling tasks like upgrades, backups, scaling, and failure recovery [8], [10].

2.4.2 Operator Development Frameworks

Kubebuilder is the most prominent framework for facilitating Operator development [15]. Built on top of controller-runtime and controller-tools libraries, it provides scaffolding for creating CRDs and controllers using Go [9]. The framework automates much of the boilerplate code generation, allowing developers to focus on implementing business logic in the reconciliation loop.

Research on Operator development best practices emphasizes the importance of proper RBAC configuration, efficient handling of custom resources, and robust error handling [14]. Studies analyzing Operator bugs across Kubernetes operators have identified common failure patterns related to state management, error handling, and race conditions [28].

2.4.3 Operator Use Cases and Applications

Operators have been successfully applied to numerous domains including database management, monitoring systems, service meshes, and machine learning platforms [8], [14]. For infrastructure provisioning, Crossplane exemplifies using

Operators to manage cloud resources declaratively through Kubernetes [13]. This approach enables treating infrastructure as Kubernetes resources, managed through the same workflows as application deployments.

2.5 Related Systems and Approaches

2.5.1 Infrastructure as Code and GitOps

Infrastructure as Code (IaC) provides declarative approaches to infrastructure provisioning [12]. While similar in declaring desired state, traditional IaC tools typically use imperative execution models and lack the continuous reconciliation capabilities inherent to Kubernetes controllers [5].

The GitOps methodology extends IaC principles by using Git as the single source of truth and implementing continuous delivery through pull-based reconciliation [19], [20]. GitOps shares philosophical alignment with Kubernetes' declarative model, making it a natural fit for managing Kubernetes resources including CRDs.

2.5.2 Service Mesh Architectures

Service mesh technologies like Istio demonstrate advanced usage of Kubernetes extensibility for managing service-to-service communication [29]. These systems use CRDs extensively to define traffic management, security policies, and observability configurations, providing examples of complex operational logic implemented through Kubernetes extension mechanisms [30].

Research comparing service mesh architectures reveals trade-offs between feature richness and operational complexity, with implications for designing custom

business logic controllers. The patterns used in service meshes offer lessons for separating concerns in business logic implementations.

2.6 Observability in Cloud-Native Systems

2.6.1 Monitoring and Metrics Collection

Observability in distributed systems encompasses metrics, logs, and traces. Prometheus has emerged as the de facto standard for metrics collection in Kubernetes environments [16], [18]. Its pull-based model, powerful query language (PromQL), and native Kubernetes service discovery make it well-suited for cloud-native monitoring [21].

For CRD-based applications, proper instrumentation of controllers and business logic is essential for understanding system behavior. Controllers should expose metrics about reconciliation loop performance, error rates, and business-specific indicators.

2.6.2 Distributed Tracing

OpenTelemetry represents the industry standard for distributed tracing and observability instrumentation [17]. Distributed tracing becomes particularly important when business logic spans multiple reconciliation loops and controllers, as it enables tracking the cascading changes of objects through the Kubernetes control plane.

The integration of observability tools with Kubebuilder-based Operators is well-established, with frameworks providing automatic instrumentation of common controller metrics. However, domain-specific business metrics require custom

implementation within the reconciliation logic.

2.7 Performance Studies and Challenges

2.7.1 Kubernetes Performance Characteristics

Studies have examined Kubernetes performance under various conditions. Research on etcd deployment impact demonstrates that control plane performance and application performance both depend critically on etcd performance characteristics [27]. Write latency and throughput limitations in etcd can become bottlenecks for CRD-intensive workloads.

2.7.2 Complexity and Operational Considerations

While Kubernetes provides powerful capabilities, it introduces significant complexity and operational overhead [1], [11]. Managing a production Kubernetes cluster requires expertise in networking, security, storage, and distributed systems. Organizations must weigh these operational costs against the benefits of Kubernetes' automation and standardization.

For business logic implementations specifically, additional complexity arises from the need to design appropriate CRD schemas, implement robust controllers, and handle edge cases in the reconciliation logic [9]. The learning curve for Kubernetes development is substantial, potentially impacting development velocity.

2.8 Research Gaps

Despite extensive research on Kubernetes architecture, CRDs, and Operators, several gaps exist in current literature:

1. Limited systematic study of using Kubernetes for general-purpose business logic implementation, with most examples focused on infrastructure or specialized domains [11].
2. Insufficient quantitative performance comparisons between CRD/Operator-based approaches and traditional architectures for equivalent business logic.
3. Lack of comprehensive guidelines on when the CRD/Operator pattern is appropriate for business applications versus when traditional architectures should be preferred.
4. Limited research on implementation complexity and developer experience when using Kubernetes as a business logic platform [11].

This thesis aims to address these gaps through systematic design, implementation, and evaluation of Kubernetes-based business logic using CRDs and Operators.

2.9 Summary

This chapter has reviewed the relevant literature on Kubernetes architecture, extensibility mechanisms, and related technologies. Kubernetes provides a powerful platform for building cloud-native applications through its declarative API, CRDs, and Operator pattern [2], [4], [8]. While extensively used for infrastructure

management, its application to general business logic remains understudied [11]. The next chapter describes the design of systems to explore this research question.

Chapter 3

Methodology

This chapter describes the research methodology, system design, and architectural decisions used to evaluate Kubernetes as a platform for implementing business logic. The chapter expands upon the high-level methodology outlined in Chapter 2 and Section 1.5, providing a detailed description of both the Kubernetes-based system and the traditional baseline system.

3.1 Research Approach

This research followed a system design and comparative analysis approach. Two functionally equivalent systems were designed and implemented:

- A Kubernetes-native system based on Custom Resource Definitions (CRDs) and the Operator pattern.
- A traditional backend system based on a REST API and relational database.

Both systems implemented the same business domain logic to ensure comparability. The primary difference lies in how the business logic is modeled, executed, and managed.

The methodology focuses on analyzing:

- Architectural differences between declarative and imperative approaches.
- Execution models (event-driven reconciliation vs request-response).
- Observability and system introspection capabilities.

3.2 Business Domain Modeling

The selected domain for this research is a billing and subscription management system. This domain was chosen because it includes both stateful entities and time-based business logic, making it suitable for evaluating different architectural paradigms.

3.2.1 Core Entities

The system defines two primary entities:

- **BillingPlan:** Represents a pricing tier, including attributes such as price, currency, billing period, and usage limits.
- **Subscription:** Represents a user's subscription to a billing plan, including references to the plan, lifecycle state, and billing timestamps.

3.2.2 Business Logic

The implemented business logic includes:

- **Activation:** Upon creation of a subscription, the system validates the referenced billing plan, activates the subscription, and initializes billing timestamps.
- **Billing Cycle Management:** The system tracks last payment and next billing dates.
- **Expiration Handling:** Subscriptions transition to an expired state when billing conditions are not met.
- **Final Charge Handling:** Upon deletion, a final billing operation is executed before resource removal.

This logic was implemented identically in both systems but using different architectural approaches.

3.3 Traditional System Architecture

The baseline system follows a conventional monolithic architecture implemented in Go.

3.3.1 Technology Stack

- **Programming language:** Go 1.25
- **HTTP framework:** Gorilla Mux (lightweight router with middleware support)
- **Database:** PostgreSQL 15 (relational database with ACID guarantees)
- **ORM:** GORM v2 (Go ORM with automatic migration support)

- **Configuration:** Viper (environment variables + YAML files)
- **Validation:** go-playground/validator v10
- **Observability:** OpenTelemetry SDK + Prometheus client_golang
- **Rate Limiting:** Token bucket algorithm (10 req/s, burst 20)

3.3.2 Architecture Overview

The system implements a layered architecture with clear separation of concerns:

- **Handlers layer:** HTTP request processing and response formatting
- **Repository layer:** Database operations through GORM interfaces
- **Models layer:** GORM entities (BillingPlan, Subscription)
- **Workers layer:** Background subscription expiration checker
- **Middleware layer:** Rate limiting, panic recovery, tracing

The REST API exposes endpoints for billing plans and subscriptions. Business logic is implemented within HTTP handlers and repository methods, with explicit error handling and validation.

3.3.3 Execution Model

The system uses a synchronous request-response model with asynchronous background processing:

- **Synchronous path:** Clients send HTTP requests → handlers validate input → repository executes database operations → response returned
- **Asynchronous path:** Background worker runs every 30 seconds, queries PostgreSQL for expired subscriptions, updates state to Expired

The background worker implements time-based business logic that cannot be handled synchronously:

```
1 // Worker runs every 30 seconds
2 ticker := time.NewTicker(30 * time.Second)
3 for range ticker.C {
4     expiredSubs := repo.FindExpired(time.Now())
5     for _, sub := range expiredSubs {
6         sub.State = "Expired"
7         repo.Update(sub)
8     }
9 }
```

Listing 3.1: Background expiration checker worker

3.3.4 Data Persistence

All system state is stored in PostgreSQL with the following schema:

- **BillingPlan table:** id (PK), name (unique), price (string for precision), currency, billing_period, created_at, updated_at, deleted_at (soft delete)
- **Subscription table:** id (PK), user_id, plan_ref (FK to BillingPlan.name), state (Active/Cancelled/Expired), last_payment, next_billing, created_at, updated_at, deleted_at

GORM auto-migration ensures schema synchronization on application startup. The database serves as the single source of truth, with business logic tightly coupled to database operations through the repository pattern.

3.3.5 API Endpoints

- GET /plans — List all billing plans
- POST /plans — Create billing plan (validates name, price, currency)
- GET /subscriptions — List all subscriptions
- GET /subscriptions/{id} — Get subscription by ID
- POST /subscriptions — Create subscription (auto-activates)
- POST /subscriptions/{id}/cancel — Cancel active subscription
- GET /health — Health check endpoint
- GET /ready — Readiness check (includes database connectivity)
- GET /metrics — Prometheus metrics endpoint

3.4 Kubernetes-Based System Architecture

The experimental system is implemented using Kubernetes as a platform for business logic through Custom Resource Definitions and the Operator pattern.

3.4.1 Technology Stack

- **Programming language:** Go 1.25.3

- **Operator Framework:** Kubebuilder v4.13.0 + controller-runtime v0.23.1
- **CRD API Group:** `billing.cloud-native.io/v1alpha1`
- **Metrics:** Prometheus client_golang v1.23.1
- **Tracing:** OpenTelemetry SDK (OTLP over HTTP, port 4318)
- **Testing:** Ginkgo v2 + Gomega + envtest (local API server)
- **E2E Testing:** Kind (Kubernetes in Docker)

3.4.2 Custom Resource Definitions

Two CRDs were defined to extend the Kubernetes API:

- `BillingPlan` (`billingplans.billing.cloud-native.io`) — defines pricing configuration
- `Subscription` (`subscriptions.billing.cloud-native.io`) — represents user subscription

BillingPlan Specification

The `BillingPlan` CRD includes comprehensive validation:

- `spec.price` (string, required): Pattern `^\d+(\.\d{1,2})?$', min length 1, max length 20` — ensures decimal with up to 2 decimal places
- `spec.currency` (string, required): Enum (USD, EUR, RUB, KZT, GBP, JPY, CNY)

- `spec.billingPeriod` (string, required): Enum (hourly, daily, weekly, monthly, yearly)
- `spec.requeueIntervalSeconds` (integer, optional): Range 1–31536000 (1 year), default 30 — controls reconciliation frequency
- `spec.limits` (map, optional): Key-value pairs for usage constraints (e.g., `apiCalls: 10000`)

Status subresource (managed by controller):

- `status.activeSubscriptions` (integer): Count of active subscriptions referencing this plan
- `status.totalRevenue` (string): Cumulative revenue from all subscriptions
- `status.conditions` (array): Standard Kubernetes conditions (Available, etc.)

Subscription Specification

The Subscription CRD enforces strict validation:

- `spec.userId` (string, required): Pattern `^[a-zA-Z0-9_-]+$`, max length 256 — alphanumeric with underscores and hyphens
- `spec.planRef` (string, required): Pattern `^[a-z0-9]([-a-z0-9]*[a-z0-9])?$', max length 253 — Kubernetes naming convention`

Status subresource (managed by controller):

- `status.state` (string): Enum (Active, PaymentError, Error)
- `status.lastPayment` (timestamp): Time of last successful billing

- `status.nextBilling` (timestamp): Scheduled time for next billing cycle
- `status.observedGeneration` (integer): Tracks status freshness against spec
- `status.conditions` (array): Standard conditions (Active, BillingPlanNotFound, PaymentError)

3.4.3 Operator Controller Implementation

The core business logic resides in the `SubscriptionReconciler`, implemented using controller-runtime. The reconciler continuously processes `Subscription` resources and ensures correct system state.

Reconciliation Workflow

The reconciliation logic follows a deterministic 6-step sequence:

1. **Fetch Subscription:** Retrieve the `Subscription` resource from the API server
2. **Handle Deletion:** If `DeletionTimestamp.IsZero() == false`:
 - Execute final billing (pro-rated refund calculation — TODO in current implementation)
 - Remove finalizer `billing.cloud-native.io/finalizer`
 - Return, allowing Kubernetes to complete deletion
3. **Add Finalizer:** If finalizer absent, attach it to prevent premature deletion
4. **Validate BillingPlan:** Query for referenced `BillingPlan` by `planRef`:
 - If not found: Set `State = Error`, `BillingPlanNotFound = True`, requeue after 60 seconds

- Emit Kubernetes Event: Warning BillingPlanNotFound

5. Initialize Subscription: If `State == ""` (new subscription):

- Set `State = Active`
- Increment Prometheus metric `billing_active_subscriptions`
- Set `LastPayment = now`, `NextBilling = now + RequeueIntervalSeconds`
- Set conditions: `Active = True`, `BillingPlanNotFound = False`, `PaymentError = False`
- Emit Event: Normal SubscriptionActivated

6. Process Billing Cycle: If `State == Active` and `now > NextBilling`:

- Increment Prometheus metric `billing_revenue_total` by plan price
- Update `LastPayment = now`, `NextBilling = now + RequeueIntervalSeconds`
- Emit Event: Normal PaymentProcessed
- **RequeueAfter:** `RequeueIntervalSeconds` from `BillingPlan.spec`

Watches and Event Sources

The controller watches multiple event sources:

- **Primary:** Subscription resources (create, update, delete events)
- **Secondary:** `BillingPlan` resources via `mapPlanToSubscriptions` function — triggers reconciliation of all Subscriptions referencing a changed Plan

This watch pattern enables automatic propagation of plan changes (e.g., price updates) to all affected subscriptions.

Finalizer Handling

A Kubernetes finalizer (billing.cloud-native.io/finalizer) ensures safe deletion:

- When user executes `kubectl delete subscription sub-user1`, Kubernetes sets `DeletionTimestamp` but does not remove the resource
- Reconciler detects deletion timestamp, executes final billing logic
- After finalizer removal, Kubernetes completes resource deletion

This mechanism guarantees cleanup logic execution even if the controller crashes during processing.

3.4.4 Execution Model

Unlike the traditional system, execution is event-driven and declarative:

- **Declarative:** User declares desired state in YAML (`spec`), controller drives actual state (`status`)
- **Event-driven:** Resource changes trigger reconciliation events via Kubernetes watch API
- **Asynchronous:** Controllers process events concurrently with configurable worker count (default: 1)

- **Eventually consistent:** System converges toward desired state through repeated reconciliation

No direct client-driven execution of business logic occurs — the controller autonomously manages state transitions.

3.4.5 Data Persistence

System state is stored in the Kubernetes control plane (etcd) as custom resources:

- **etcd:** Distributed key-value store (backing Kubernetes API server)
- **CRD instances:** YAML resources persisted with full versioning and meta-data
- **Status subresource:** Separate update path for controller-managed state (optimizes write conflicts)

The declarative state model enables automatic recovery: if the controller crashes, it reconstructs state from etcd on restart and resumes reconciliation.

3.4.6 Observability Integration

Prometheus Metrics

Three custom business metrics are exposed at `:8080/metrics`:

- `billing_active_subscriptions` (Gauge): Current count of Active subscriptions
- `billing_revenue_total` (Counter): Cumulative revenue processed (sum of all successful billing cycles)

- `billing_payment_failures` (Counter): Number of failed payment attempts (defined but not incremented in current implementation)

Additionally, `controller-runtime` exposes standard metrics:

- `controller_runtime_reconcile_total` (Counter): Total reconciliation attempts by result (success, error, requeue)
- `controller_runtime_reconcile_errors_total` (Counter): Total reconciliation errors

OpenTelemetry Tracing

Distributed tracing captures asynchronous reconciliation flow:

- **Root span:** `ReconcileSubscription` — covers entire reconciliation cycle
- **Child span:** `ProcessPayment` — billing logic execution
- **Mapped span:** `MapPlanToSubscriptions` — triggered by `BillingPlan` watch events

Traces are exported via OTLP over HTTP to `localhost:4318` (Jaeger-compatible).

Kubernetes Events

The controller emits audit events for significant state changes:

- Normal `SubscriptionActivated`: Subscription successfully activated
- Normal `PaymentProcessed`: Billing cycle completed
- Warning `PaymentFailed`: Payment processing failed

- Normal FinalBilling: Final billing executed on deletion

Events are accessible via `kubectl get events --field-selector involvedObject.name=subuser1`.

3.4.7 Deployment Configuration

The system is deployed using production-ready Kubernetes manifests:

- **HorizontalPodAutoscaler (HPA)**: Min 1, Max 5 replicas, CPU target 80%, Memory target 80%
- **PodDisruptionBudget (PDB)**: Min available 1 — ensures availability during voluntary disruptions
- **Resource limits**: CPU 100m–500m, Memory 128Mi–256Mi
- **NetworkPolicy**: Restricts `/metrics` endpoint access to Prometheus namespace
- **RBAC**: ServiceAccount with minimal permissions (get, list, watch Subscriptions and BillingPlans)

Deployment is managed via Kustomize overlays (`config/default`) combining CRDs, RBAC, controller Deployment, and monitoring configuration.

3.5 Comparison of Architectural Approaches

The two systems differ fundamentally in how they implement business logic. Table 3.1 summarizes the key architectural differences.

TABLE 3.1
Architectural Comparison: Traditional Backend vs Kubernetes Operator

Aspect	backend-billing	kube-billing
State storage	PostgreSQL (external)	etcd via CRD (control plane)
API style	REST HTTP (port 8080)	Kubernetes API (kubectl)
Execution model	Request-response + background worker	Reconcile loop + RequeueAfter
Scaling	Manual (Docker Compose)	HPA (automatic, 1–5 replicas)
Fault tolerance	Depends on DB + orchestrator	Kubernetes self-healing
Validation	Runtime (go-playground/validator)	Admission time (CRD schema)
Error handling	Explicit (error returns)	Eventual consistency + retries
Observability	HTTP metrics + DB tracing	Business metrics + Events + Conditions
Deployment	Docker container	Kubernetes Deployment + HPA + PDB
Configuration	Environment variables + YAML	ConfigMap + Secrets

3.5.1 State Management

backend-billing stores state in PostgreSQL, an external relational database.

This approach provides:

- Strong ACID guarantees for transactions
- Mature query capabilities (JOINS, aggregations, indexes)
- Separation of application and data layers

kube-billing stores state in etcd via Custom Resources. This approach provides:

- Declarative state management (spec vs status separation)
- Built-in versioning and metadata (generation, resourceVersion)
- Automatic persistence through Kubernetes control plane

3.5.2 Execution Flow

backend-billing follows imperative execution:

```
1 POST /subscriptions      validate      INSERT      201 Created
```

Listing 3.2: Traditional REST API request flow

kube-billing follows declarative reconciliation:

```
1 kubectl apply -f sub.yaml      etcd write      watch event      Reconcile  
   ()      status update
```

Listing 3.3: Kubernetes Operator reconciliation flow

3.5.3 Time-Based Logic

backend-billing uses a ticker-based worker:

- Fixed interval (30 seconds)
- Polls database for expired subscriptions
- Updates state synchronously within worker goroutine

kube-billing uses `RequeueAfter`:

- Dynamic interval from `BillingPlan.spec.requeueIntervalSeconds`
- Controller-runtime schedules next reconciliation
- No polling — event-driven wake-up

3.5.4 Lifecycle Management

backend-billing deletes immediately:

- DELETE /subscriptions/{id} → soft delete (deleted_at timestamp)
- No cleanup hook execution guarantee

kube-billing uses finalizers:

- kubectl delete → DeletionTimestamp set → finalizer blocks deletion
- Reconciler executes final billing → removes finalizer → Kubernetes deletes
- Guaranteed cleanup even on controller failure

3.6 Observability Design

Observability was implemented in both systems to enable comparative analysis of system behavior, with emphasis on business-level events rather than only infrastructure metrics.

3.6.1 Kubernetes System Observability

The Kubernetes-based system integrates three observability layers:

Prometheus Metrics

Custom business metrics exposed at :8080/metrics:

- billing_active_subscriptions (Gauge) — current count of Active subscriptions
- billing_revenue_total (Counter) — cumulative revenue from successful billing cycles

- `billing_payment_failures` (Counter) — failed payment attempts (defined, not incremented)

Standard controller-runtime metrics:

- `controller_runtime_reconcile_total` — reconciliation attempts by result
- `controller_runtime_reconcile_errors_total` — error count
- `workqueue_depth` — pending work items
- `process_cpu_seconds`, `go_memstats_alloc_bytes` — runtime metrics

OpenTelemetry Tracing

Span hierarchy for reconciliation:

- `ReconcileSubscription` (root) — entire reconciliation cycle
- `ProcessPayment` (child) — billing logic execution
- `MapPlanToSubscriptions` (child) — triggered by `BillingPlan` watch

Export: OTLP over HTTP to `localhost:4318` (Jaeger-compatible).

Kubernetes Events

Audit trail via Events API:

- Normal `SubscriptionActivated` — new subscription activated
- Normal `PaymentProcessed` — billing cycle completed
- Warning `PaymentFailed` — payment error
- Normal `FinalBilling` — final billing on deletion

3.6.2 Traditional System Observability

The traditional system implements two-layer observability:

Prometheus Metrics

HTTP and business metrics at `/metrics`:

- `billing_subscriptions_created_total` (Counter)
- `billing_subscriptions_cancelled_total` (Counter)
- `http_request_duration_seconds` (Histogram) — request latency
- `http_requests_total` (Counter) — request count by method/path

OpenTelemetry Tracing

Integrated tracing for HTTP and database layers:

- HTTP middleware (`otelmux`) — traces request lifecycle
- GORM plugin — traces SQL queries (duration, statement, rows affected)

Export: OTLP over HTTP to Jaeger (<http://jaeger:4318>).

3.6.3 Comparative Analysis

The Kubernetes system provides richer business-level observability:

- **Events** — audit trail without custom logging
- **Conditions** — standardized state reporting in resource status
- **Reconciliation metrics** — visibility into control loop behavior

The traditional system provides simpler but more direct observability:

- **HTTP metrics** — immediate request/response visibility
- **Database tracing** — direct query-level insights

3.7 Experimental Methodology

Experimental evaluation was conducted in a controlled local environment to compare the two architectural approaches across multiple dimensions.

3.7.1 Evaluation Environment

Both systems were deployed and tested on identical hardware:

- **CPU:** Apple M4 (single-core performance isolated for benchmarks)
- **RAM:** 16GB unified memory
- **OS:** macOS (Darwin kernel)
- **Kubernetes:** Minikube v1.33.1 (for kube-billing)
- **Docker:** Docker Desktop v27.5.0 (for backend-billing)

3.7.2 Evaluation Metrics

The evaluation focuses on four categories:

Code Metrics (Lean Software Development)

- **Lines of Code (LOC)** — total Go source lines (excluding tests)
- **File count** — number of Go source files
- **Cyclomatic complexity** — maximum complexity per function
- **Test coverage** — percentage of code covered by automated tests
- **Dependencies** — number of Go module dependencies

Performance Metrics

- **Latency** — p50, p95, p99 response times for CRUD operations
- **Throughput** — requests per second under load
- **Resource usage** — CPU (millicores) and memory (MB) consumption
- **Startup time** — time from launch to readiness

Operational Metrics

- **Scaling** — manual vs automatic (HPA)
- **Fault tolerance** — recovery from controller/database failure
- **Deployment complexity** — number of configuration files, steps

Observability Coverage

- **Metrics count** — number of custom business metrics
- **Trace coverage** — percentage of code paths traced

- **Event/audit trail** — presence of audit logging

3.7.3 Benchmark Methodology

Load Testing

backend-billing:

```
1 hey -n 1000 -c 10 http://localhost:8080/plans
2 hey -n 1000 -c 50 http://localhost:8080/subscriptions
```

Listing 3.4: Load testing commands for backend-billing

kube-billing:

```
1 for i in {1..1000}; do kubectl apply -f plan.yaml; done
2 kubectl apply -f subscription.yaml (sequential, rate-limited)
```

Listing 3.5: Load testing commands for kube-billing

Resource Monitoring

- **backend-billing:** docker stats, Go pprof
- **kube-billing:** kubectl top pods, Prometheus metrics

Microbenchmarks

Go testing package for isolated function benchmarks:

```
1 go test -bench=BenchmarkCreatePlan -benchmem
2 go test -bench=BenchmarkValidatePlan -benchmem
```

Listing 3.6: Go microbenchmark commands

3.7.4 Threats to Validity

Several limitations affect the generalizability of results:

- **Local environment** — results may differ in production Kubernetes clusters with network latency and resource contention
- **Single-node testing** — no distributed system effects (network partitions, leader election)
- **Simulated payments** — no external payment gateway integration (latency, failures)
- **Short test duration** — no long-term etcd compaction or database bloat effects

Despite these limitations, the comparative analysis provides meaningful insights into architectural trade-offs.

3.8 Summary

This chapter described the methodology used to evaluate Kubernetes as a platform for business logic. Two systems implementing identical functionality were designed using different paradigms: a traditional REST-based system with PostgreSQL and a Kubernetes-native Operator-based system with CRDs. The architectural differences — imperative vs declarative, request-response vs reconciliation, external database vs etcd — provide the foundation for the comparative evaluation presented in Chapter 5.

Chapter 4

Implementation

This chapter describes the implementation of the two systems introduced in Chapter 3. Both systems were developed in Go to ensure consistency in comparison, but differ significantly in architecture, execution model, and state management. The goal is to provide detailed insight into implementation decisions, internal logic, and system behavior.

4.1 Implementation Overview

Both systems implement identical business logic for billing and subscription management:

- **BillingPlan:** Pricing configuration with price, currency, billing period
- **Subscription:** User subscription linked to a plan with lifecycle management
- **Billing cycle:** Automatic recurring billing with configurable intervals
- **Expiration handling:** State transitions when billing conditions are not met
- **Final charge:** Cleanup logic on subscription deletion

The implementations differ in:

- **State persistence:** PostgreSQL vs etcd via CRD
- **Execution model:** Request-response vs reconcile loop
- **Validation:** Runtime validation vs admission control
- **Observability:** HTTP/DB tracing vs Events/Conditions/Reconciliation metrics

4.2 Kubernetes-Based System (kube-billing)

4.2.1 Project Structure

The Kubernetes-based system was implemented using Kubebuilder v4.13.0, which scaffolds the project structure and generates boilerplate code.

```

1 kube-billing /
2     cmd/main.go # Entry point, manager setup
3     api/v1alpha1 /
4         billingplan_types.go # BillingPlan CRD spec/status
5         subscription_types.go # Subscription CRD spec/status
6         groupversion_info.go # GroupVersion for scheme
7     internal/controller /
8         subscription_controller.go # Reconcile logic
9         metrics.go # Prometheus metrics
10        tracing.go # OpenTelemetry setup
11    config /
12        crd/bases / # Generated CRD YAML
13        rbac / # ServiceAccount, Role,
14        RoleBinding
15        manager / # Deployment manifest
16        samples / # Example CRs
17        default / # Kustomize overlay (HPA, PDB)

```

```

17      test/e2e/                                # E2E tests with Kind
18      Makefile                                  # Build, test, deploy targets
19      Dockerfile                               # Multi-stage build

```

Listing 4.1: kube-billing project structure

4.2.2 CRD Implementation

BillingPlan CRD

The BillingPlan CRD is defined in `api/v1alpha1/billingplan_types.go`:

```

1  type BillingPlan struct {
2      metav1.TypeMeta   'json:",inline"'
3      metav1.ObjectMeta 'json:"metadata,omitempty"'
4      Spec               BillingPlanSpec   'json:"spec,omitempty"'
5      Status             BillingPlanStatus 'json:"status,omitempty"'
6  }
7
8  type BillingPlanSpec struct {
9      Price                string          'json:"price"'
10     Currency              Currency        'json:"currency"'
11     BillingPeriod          BillingPeriod    'json:"billingPeriod"'
12     RequeueIntervalSeconds int             'json:"
13     requeueIntervalSeconds,omitempty"'
14     Limits                 map[string]int64 'json:"limits,omitempty"'
15 }

```

Validation markers (kubebuilder annotations):

```

1  // +kubebuilder:validation:Pattern='^\d+(\.\d{1,2})?$',
2  // +kubebuilder:validation:MinLength=1
3  // +kubebuilder:validation:MaxLength=20
4  Price string
5
6  // +kubebuilder:validation:Enum=USD;EUR;RUB;KZT;GBP;JPY;CNY
7  Currency string

```

```

8
9 // +kubebuilder:validation:Enum=hourly;daily;weekly;monthly;yearly
10 BillingPeriod string
11
12 // +kubebuilder:validation:Minimum=1
13 // +kubebuilder:validation:Maximum=31536000
14 RequeueIntervalSeconds int

```

Status subresource (managed by controller):

```

1 type BillingPlanStatus struct {
2     ActiveSubscriptions int    'json:"activeSubscriptions,omitempty"'
3     TotalRevenue        string 'json:"totalRevenue,omitempty"'
4     Conditions          []metav1.Condition 'json:"conditions,omitempty"'
5     '
6 }

```

Subscription CRD

The Subscription CRD is defined in `api/v1alpha1/subscription_types.go`:

```

1 type Subscription struct {
2     metav1.TypeMeta    'json:",inline"'
3     metav1.ObjectMeta   'json:"metadata,omitempty"'
4     Spec                SubscriptionSpec    'json:"spec,omitempty"'
5     Status              SubscriptionStatus  'json:"status,omitempty"'
6 }
7
8 type SubscriptionSpec struct {
9     UserID string 'json:"userId"'
10    PlanRef string 'json:"planRef"'
11 }
12
13 // +kubebuilder:validation:Pattern='^[a-zA-Z0-9_-]+$'
14 // +kubebuilder:validation:MaxLength=256
15 UserID string
16

```

```

17 // +kubebuilder:validation:Pattern='^[a-z0-9]([-a-z0-9]*[a-z0-9])?$'
18 // +kubebuilder:validation:MaxLength=253
19 PlanRef string

```

Status subresource:

```

1 type SubscriptionStatus struct {
2     State SubscriptionState 'json:"state,omitempty"'
3     LastPayment *metav1.Time 'json:"lastPayment,omitempty"'
4     NextBilling *metav1.Time 'json:"nextBilling,omitempty"'
5     ObservedGeneration int64 'json:"observedGeneration,omitempty"'
6     Conditions []metav1.Condition 'json:"conditions,omitempty"'
7 }
8
9 // +kubebuilder:validation:Enum=Active;PaymentError;Error
10 type SubscriptionState string

```

4.2.3 Controller Implementation

The core business logic resides in SubscriptionReconciler (internal/controller/subscriptionreconciler.go).

Reconcile Method

The Reconcile method implements the 6-step workflow:

```

1 func (r *SubscriptionReconciler) Reconcile(ctx context.Context,
2     req ctrl.Request) (ctrl.Result, error) {
3
4     // Step 1: Fetch Subscription
5     var sub Subscription
6     if err := r.Get(ctx, req.NamespacedName, &sub); err != nil {
7         return ctrl.Result{}, client.IgnoreNotFound(err)
8     }
9
10    // Step 2: Handle deletion

```



```

11     if !sub.DeletionTimestamp.IsZero() {
12         return r.handleDeletion(ctx, &sub)
13     }
14
15     // Step 3: Add finalizer
16     if !controllerutil.ContainsFinalizer(&sub, FinalizerName) {
17         controllerutil.AddFinalizer(&sub, FinalizerName)
18         return r.Update(ctx, &sub)
19     }
20
21     // Step 4: Validate BillingPlan
22     plan, err := r.getPlan(ctx, sub.Spec.PlanRef)
23     if err != nil {
24         return r.setErrorCondition(ctx, &sub, err)
25     }
26
27     // Step 5: Initialize if new
28     if sub.Status.State == "" {
29         return r.activateSubscription(ctx, &sub, plan)
30     }
31
32     // Step 6: Process billing cycle
33     if sub.Status.State == StateActive && time.Now().After(sub.Status.
NextBilling.Time) {
34         return r.processPayment(ctx, &sub, plan)
35     }
36
37     return ctrl.Result{RequeueAfter: time.Duration(plan.Spec.
RequeueIntervalSeconds) * time.Second}, nil
38 }

```

Finalizer Handling

The finalizer `billing.cloud-native.io/finalizer` ensures safe deletion:

```

1 func (r *SubscriptionReconciler) handleDeletion(ctx context.Context,

```

```

2   sub *Subscription) (ctrl.Result, error) {
3
4   // Execute final billing (pro-rated refund)
5   // TODO: Implement refund calculation
6
7   // Remove finalizer
8   controllerutil.RemoveFinalizer(sub, FinalizerName)
9   return ctrl.Result{}, r.Update(ctx, sub)
10 }

```

Watches and Event Mapping

The controller watches both Subscriptions and BillingPlans:

```

1 func (r *SubscriptionReconciler) SetupWithManager(mgr ctrl.Manager)
   error {
2     return ctrl.NewControllerManagedBy(mgr).
3         For(&billingv1alpha1.Subscription{}).
4         Watches(
5             &billingv1alpha1.BillingPlan{},
6             handler.EnqueueRequestsFromMapFunc(r.mapPlanToSubscriptions
7         ),
8         ).
9         Complete(r)
10 }
11 func (r *SubscriptionReconciler) mapPlanToSubscriptions(ctx context.
   Context,
12     obj client.Object) []reconcile.Request {
13
14     plan := obj.(*billingv1alpha1.BillingPlan)
15     var subs SubscriptionList
16     r.List(ctx, &subs, client.MatchingFields{"spec.planRef": plan.Name
17     })
18     var requests []reconcile.Request

```

```
19     for _, sub := range subs.Items {
20         requests = append(requests, reconcile.Request{
21             NamespacedName: types.NamespacedName{
22                 Namespace: sub.Namespace,
23                 Name:       sub.Name,
24             },
25         })
26     }
27     return requests
28 }
```

4.2.4 Metrics Implementation

Custom Prometheus metrics are defined in `internal/controller/metrics.go`:

```
1 var (
2     activeSubscriptions = prometheus.NewGauge(prometheus.GaugeOpts{
3         Name: "billing_active_subscriptions",
4         Help: "Current number of active subscriptions",
5     })
6
7     revenueTotal = prometheus.NewCounter(prometheus.CounterOpts{
8         Name: "billing_revenue_total",
9         Help: "Total revenue processed",
10    })
11
12    paymentFailures = prometheus.NewCounter(prometheus.CounterOpts{
13        Name: "billing_payment_failures",
14        Help: "Number of failed payment attempts",
15    })
16 )
17
18 func init() {
19     reg.MustRegister(activeSubscriptions, revenueTotal, paymentFailures)
20 }
```

Metrics are exposed at :8080/metrics and scraped by Prometheus via ServiceMonitor.

4.2.5 Tracing Implementation

OpenTelemetry tracing is initialized in internal/controller/tracing.go:

```
1 func InitTracer(ctx context.Context) (func(), error) {
2     exporter, err := otlptracehttp.New(ctx,
3         otlptracehttp.WithEndpoint("localhost:4318"),
4         otlptracehttp.WithInsecure(),
5     )
6
7     tp := trace.NewTracerProvider(
8         trace.WithBatcher(exporter),
9         trace.WithResource(resource.NewWithAttributes(
10             semconv.ServiceNameKey.String("kube-billing"),
11         )),
12     )
13
14     otel.SetTracerProvider(tp)
15     return func() { tp.Shutdown(ctx) }, nil
16 }
```

Spans are created in the reconcile loop:

```
1 ctx, span := otel.Tracer("kube-billing").Start(ctx, "
    ReconcileSubscription")
2 defer span.End()
3
4 // Payment processing span
5 _, paymentSpan := otel.Tracer("kube-billing").Start(ctx, "
    ProcessPayment")
6 defer paymentSpan.End()
```

4.2.6 Deployment Configuration

Production deployment includes HPA, PDB, and resource limits:

```
1 # config/default/hpa.yaml
2 apiVersion: autoscaling/v2
3 kind: HorizontalPodAutoscaler
4 metadata:
5   name: kube-billing-hpa
6 spec:
7   scaleTargetRef:
8     apiVersion: apps/v1
9     kind: Deployment
10    name: kube-billing
11  minReplicas: 1
12  maxReplicas: 5
13  metrics:
14  - type: Resource
15    resource:
16      name: cpu
17      target:
18        type: Utilization
19        averageUtilization: 80
20  ---
21 # config/default/pdb.yaml
22 apiVersion: policy/v1
23 kind: PodDisruptionBudget
24 metadata:
25   name: kube-billing-pdb
26 spec:
27   minAvailable: 1
28   selector:
29     matchLabels:
30       control-plane: controller-manager
```

Listing 4.2: Kubernetes HPA and PDB configuration

4.3 Traditional System (backend-billing)

4.3.1 Project Structure

The traditional backend follows a layered architecture:

```
1 backend-billing /
2     main.go # Entry point, router setup
3     config /
4         config.go # Viper configuration
5         prometheus.yml # Prometheus scrape config
6     db /
7         db.go # GORM initialization ,
8     migrations
9     handlers /
10        plan_handlers.go # HTTP handlers for BillingPlan
11        subscription_handlers.go # HTTP handlers for
12        Subscription
13        health_handlers.go # Health check endpoints
14    middleware /
15        rate_limiter.go # Token bucket rate limiting
16        recovery.go # Panic recovery
17    models /
18        models.go # GORM models
19    repository /
20        repository.go # Repository interfaces
21        plan_repository.go # Plan CRUD operations
22        subscription_repository.go # Subscription CRUD
23    workers /
24        workers.go # Background expiration checker
25    validator /
26        validator.go # Custom validation rules
27    apierrors /
28        errors.go # Typed API errors
29    observability /
30        setup.go # Logger and tracer setup
31    tests /
```

```

30         integration/           # Integration tests (
    testcontainers)
31         e2e/                   # E2E tests (full HTTP scenarios
    )
32     docker-compose.yml         # PostgreSQL, Jaeger, Prometheus

```

Listing 4.3: backend-billing project structure

4.3.2 Data Models

GORM models are defined in models/models.go:

```

1  type BillingPlan struct {
2      ID          uint           'gorm:"primaryKey"'
3      Name        string        'gorm:"uniqueIndex"'
4      Price       string        'gorm:"type:decimal(10,2)"'
5      Currency    string        'gorm:"size:3"'
6      BillingPeriod string      'gorm:"size:10"'
7      CreatedAt   time.Time
8      UpdatedAt   time.Time
9      DeletedAt   gorm.DeletedAt 'gorm:"index"'
10 }
11
12 type Subscription struct {
13     ID          uint           'gorm:"primaryKey"'
14     UserID       string        'gorm:"size:255;index"'
15     PlanRef      string        'gorm:"size:255;index"'
16     State        string        'gorm:"size:20;default:'Active'''
17     LastPayment  *time.Time
18     NextBilling  *time.Time
19     CreatedAt   time.Time
20     UpdatedAt   time.Time
21     DeletedAt   gorm.DeletedAt 'gorm:"index"'
22 }

```

4.3.3 HTTP Handlers

Subscription handler implements CRUD operations:

```
1 // handlers/subscription_handlers.go
2 func (h *SubscriptionHandler) CreateSubscription(w http.ResponseWriter,
3     r *http.Request) {
4     var req CreateSubscriptionRequest
5     if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
6         http.Error(w, err.Error(), http.StatusBadRequest)
7         return
8     }
9
10    // Validate
11    if err := validator.Validate(req); err != nil {
12        apierrors.WriteError(w, apierrors.ErrValidationFailed(err))
13        return
14    }
15
16    // Create
17    sub, err := h.repo.Create(r.Context(), &Subscription{
18        UserID: req.UserID,
19        PlanRef: req.PlanRef,
20        State:   "Active",
21    })
22
23    if err != nil {
24        apierrors.WriteError(w, apierrors.ErrDatabase(err))
25        return
26    }
27
28    // Metrics
29    metrics.SubscriptionsCreated.Inc()
30
31    w.WriteHeader(http.StatusCreated)
32    json.NewEncoder(w).Encode(sub)
33 }
```


4.3.4 Repository Layer

Repository pattern abstracts database operations:

```
1 // repository/subscription_repository.go
2 type SubscriptionRepository interface {
3     Create(ctx context.Context, sub *models.Subscription) (*models.
4     Subscription, error)
5     GetByID(ctx context.Context, id uint) (*models.Subscription, error)
6     List(ctx context.Context) ([]*models.Subscription, error)
7     Cancel(ctx context.Context, id uint) error
8     FindExpired(ctx context.Context, now time.Time) ([]*models.
9     Subscription, error)
10    Update(ctx context.Context, sub *models.Subscription) error
11 }
12
13 type gormSubscriptionRepo struct {
14     db *gorm.DB
15 }
16
17 func (r *gormSubscriptionRepo) FindExpired(ctx context.Context, now
18     time.Time) ([]*models.Subscription, error) {
19     var subs []*models.Subscription
20     err := r.db.WithContext(ctx).
21         Where("state = ? AND next_billing < ?", "Active", now).
22         Find(&subs).Error
23     return subs, err
24 }
```

4.3.5 Background Worker

The expiration checker runs every 30 seconds:

```
1 // workers/workers.go
2 func StartExpirationChecker(ctx context.Context, repo
3     SubscriptionRepository, interval time.Duration) {
4     ticker := time.NewTicker(interval)
```

```
4     defer ticker.Stop()
5
6     for {
7         select {
8             case <-ctx.Done():
9                 return
10            case <-ticker.C:
11                expired, err := repo.FindExpired(ctx, time.Now())
12                if err != nil {
13                    log.Error("Failed to find expired", "error", err)
14                    continue
15                }
16
17                for _, sub := range expired {
18                    sub.State = "Expired"
19                    if err := repo.Update(ctx, sub); err != nil {
20                        log.Error("Failed to update", "id", sub.ID, "error"
21, err)
22                    }
23                }
24                log.Info("Expiration check completed", "expired", len(
25expired))
26            }
27        }
```

4.3.6 Validation

Input validation uses go-playground/validator:

```
1 // validator/validator.go
2 var validate *validator.Validate
3
4 func init() {
5     validate = validator.New()
6 }
```

```
7 // Custom validation for currency
8 validate.RegisterValidation("currency", func(fl validator.
FieldLevel) bool {
9     currency := fl.Field().String()
10    validCurrencies := map[string]bool{
11        "USD": true, "EUR": true, "RUB": true,
12        "KZT": true, "GBP": true, "JPY": true, "CNY": true,
13    }
14    return validCurrencies[currency]
15 })
16 }
17
18 type CreateSubscriptionRequest struct {
19     UserID string `validate:"required,alphanum,max=255"`
20     PlanRef string `validate:"required,max=255"`
21 }
```

4.3.7 Observability

Metrics

Prometheus metrics in handlers/subscription_handlers.go:

```
1 var (
2     subscriptionsCreated = prometheus.NewCounter(prometheus.CounterOpts
3     {
4         Name: "billing_subscriptions_created_total",
5         Help: "Total number of created subscriptions",
6     })
7
8     subscriptionsCancelled = prometheus.NewCounter(prometheus.
9     CounterOpts{
10         Name: "billing_subscriptions_cancelled_total",
11         Help: "Total number of cancelled subscriptions",
12     })
13 )
```

Tracing

OpenTelemetry integration in observability/setup.go:

```

1 func SetupTracing(ctx context.Context) (func(), error) {
2     exporter, err := otlptracehttp.New(ctx,
3         otlptracehttp.WithEndpoint("jaeger:4318"),
4         otlptracehttp.WithInsecure(),
5     )
6
7     tp := trace.NewTracerProvider(
8         trace.WithBatcher(exporter),
9         trace.WithResource(resource.NewWithAttributes(
10             semconv.ServiceNameKey.String("backend-billing"),
11         )),
12     )
13
14     otel.SetTracerProvider(tp)
15
16     // Apply to HTTP router
17     router.Use(otelmux.Middleware("billing-api"))
18
19     // Apply to GORM
20     db.Use(tracing.NewPlugin())
21
22     return func() { tp.Shutdown(ctx) }, nil
23 }

```

4.4 Implementation Comparison

4.4.1 Code Metrics

Table 4.1 compares code metrics between the two implementations.

Analysis:

TABLE 4.1
Code Metrics Comparison

Metric	backend-billing	kube-billing
Go source files	24	13
Lines of code (excluding tests)	3,061	2,483
Test files	8	4
Max cyclomatic complexity	12	25
Dependencies (go.mod)	214	191
Test coverage	86.2%	~70%

- **backend-billing** has more files but lower complexity — traditional separation of concerns
- **kube-billing** is more concentrated — reconcile loop handles multiple responsibilities
- Higher complexity in kube-billing due to Reconcile pattern and Kubernetes API integration

4.4.2 State Management

backend-billing:

- Explicit CRUD operations through repository pattern
- Transactions via GORM (db.Begin(), Commit(), Rollback())
- Soft delete via DeletedAt timestamp

kube-billing:

- Declarative state via spec (desired) and status (actual)
- Optimistic locking via resourceVersion

- Finalizers for cleanup guarantees
- Conditions for standardized state reporting

4.4.3 Error Handling

backend-billing:

```
1 // Explicit error returns
2 sub, err := h.repo.Create(ctx, &Subscription {...})
3 if err != nil {
4     if errors.Is(err, repository.ErrNotFound) {
5         apierrors.WriteError(w, apierrors.ErrNotFound("plan"))
6     } else {
7         apierrors.WriteError(w, apierrors.ErrDatabase(err))
8     }
9     return
10 }
```

kube-billing:

```
1 // Eventual consistency with requeue
2 plan, err := r.getPlan(ctx, sub.Spec.PlanRef)
3 if err != nil {
4     // Set error condition
5     sub.Status.Conditions = append(sub.Status.Conditions,
6         metav1.Condition{
7             Type: "BillingPlanNotFound",
8             Status: "True",
9             Reason: "PlanNotFound",
10        })
11    r.Status().Update(ctx, sub)
12
13    // Requeue for retry
14    return ctrl.Result{RequeueAfter: 60 * time.Second}, nil
15 }
```

4.4.4 Testing Approach

backend-billing:

- **Unit tests:** testify mocks for repository layer
- **Integration tests:** testcontainers-go (PostgreSQL in Docker)
- **E2E tests:** HTTP client against running server

kube-billing:

- **Unit tests:** envtest (local API server + etcd)
- **Integration tests:** Ginkgo + Gomega with fake client
- **E2E tests:** Kind cluster (Kubernetes in Docker)

4.5 Implementation Challenges

4.5.1 Kubernetes System Challenges

- **Steep learning curve:** Kubebuilder, controller-runtime, CRD schema
- **Debugging complexity:** Asynchronous reconcile loop, eventual consistency
- **Schema evolution:** CRD versioning and migration strategies
- **Testing overhead:** envtest setup, Kind cluster management

4.5.2 Traditional System Challenges

- **Manual lifecycle management:** No built-in finalizer equivalent

- **Polling overhead:** Background worker continuously queries database
- **Scaling complexity:** Manual horizontal scaling with load balancer
- **State recovery:** No automatic reconstruction from declarative state

4.6 Summary

This chapter presented detailed implementation of both systems. The Kubernetes-based system leverages CRDs, Operators, and the reconcile pattern to implement declarative business logic. The traditional system uses REST APIs, PostgreSQL, and background workers for imperative logic. Key differences include state management (etcd vs PostgreSQL), execution model (event-driven vs request-response), and error handling (eventual consistency vs explicit errors). These implementation details form the basis for the experimental evaluation in Chapter 5.

Chapter 5

Evaluation and Discussion

This chapter presents the experimental evaluation of the two billing system implementations described in Chapters 3 and 4. The evaluation compares the Kubernetes-based Operator approach against the traditional REST API backend across multiple dimensions: code metrics, performance, resource utilization, and operational characteristics.

5.1 Experimental Setup

5.1.1 Test Environment

All experiments were conducted on identical hardware to ensure fair comparison:

- **CPU:** Apple M4 (8-core, 3.0 GHz)
- **RAM:** 16 GB unified memory
- **OS:** macOS Sonoma 14.0 (Darwin kernel 23.0)

- **Go version:** 1.25 (backend-billing), 1.25.3 (kube-billing)

backend-billing environment:

- Docker Desktop v27.5.0
- PostgreSQL 15 (via docker-compose)
- Jaeger 1.50 (tracing backend)
- Prometheus 2.45 (metrics collection)

kube-billing environment:

- Minikube v1.33.1 (Kubernetes v1.29.0)
- containerd 1.7.0 (container runtime)
- Metrics Server 0.7.0 (kubectl top)
- Prometheus Operator (metrics scraping)

5.1.2 Benchmark Tools

- **hey** v0.0.1 — HTTP load generator (backend-billing)
- **kubectl** v1.29.0 — Kubernetes CLI (kube-billing)
- **go test -bench** — Go microbenchmarks
- **docker stats** — Container resource monitoring
- **kubectl top pods** — Kubernetes resource monitoring

5.2 Code Metrics Comparison

5.2.1 Lines of Code and File Count

Table 5.1 shows the code size metrics for both implementations.

TABLE 5.1
Code Size Metrics

Metric	backend-billing	kube-billing
Go source files	24	13
Lines of code (total)	3,061	2,483
Test files	8	4
Average LOC per file	127	191

Analysis: The Kubernetes implementation is 19% smaller in total LOC but has 50% higher density (LOC/file). This reflects the concentrated nature of reconcile logic versus the layered architecture of the traditional backend.

5.2.2 Cyclomatic Complexity

Cyclomatic complexity measures the number of linearly independent paths through code.

TABLE 5.2
Cyclomatic Complexity

Metric	backend-billing	kube-billing
Maximum complexity	12	25
Average complexity	4.2	8.7
Functions with complexity > 10	2	5

Analysis: The Kubernetes controller has 108% higher maximum complexity due to the reconcile loop handling multiple state transitions, error conditions, and requeue logic. This complexity is inherent to the Operator pattern.

5.2.3 Dependencies

TABLE 5.3
Go Module Dependencies

Metric	backend-billing	kube-billing
Direct dependencies	45	38
Transitive dependencies	214	191
Total module size	42 MB	38 MB

Analysis: Both systems have comparable dependency footprints. The Kubernetes system benefits from controller-runtime bundling many utilities, while the traditional system requires more explicit dependencies (GORM, Gorilla Mux, validator).

5.2.4 Test Coverage

TABLE 5.4
Test Coverage

Metric	backend-billing	kube-billing
Statement coverage	86.2%	~70%
Integration tests	12	6
E2E tests	8	4

Analysis: The traditional backend has higher test coverage due to simpler mocking (interface-based repositories) versus Kubernetes client mocking. E2E tests for kube-billing require Kind cluster setup, increasing test complexity.

5.3 Performance Evaluation

5.3.1 Microbenchmarks

Go microbenchmarks measure isolated function performance without I/O overhead.

TABLE 5.5
Microbenchmark Results (Apple M4)

Benchmark	Time/op	Memory/op	Allocs/op
CreatePlan (validation)	855 ns	5,171 B	11
ValidatePlan	359 ns	368 B	10
WriteError	649 ns	1,016 B	10
HealthCheck	1,383 ns	6,184 B	20

Analysis: Pure Go operations complete in sub-microsecond timeframes with minimal allocations. These benchmarks represent best-case latency for the traditional backend without database or network overhead.

5.3.2 End-to-End Latency

Table 5.6 shows realistic latency including I/O.

TABLE 5.6
End-to-End Latency (p50/p95/p99 in ms)

Operation	backend-billing		kube-billing	
	p50/p95	p99	p50/p95	p99
Create Plan	5 / 12	25	50 / 120	250
Create Subscription	10 / 25	50	100 / 200	400
Get Subscription	3 / 8	15	30 / 80	150
Cancel Subscription	8 / 20	40	80 / 180	350

Analysis: The traditional backend is 10–20× faster for all operations. The Kubernetes API server introduces overhead from:

- Admission control (validation webhooks)
- etcd write latency (distributed consensus)
- Resource versioning and conflict detection
- Watch event propagation

5.3.3 Throughput Under Load

Load testing with hey (backend-billing) and sequential kubectl apply (kube-billing):

TABLE 5.7
Throughput Comparison

Metric	backend-billing	kube-billing
Requests per second	~1,000	~200
Concurrent connections	500	100
Error rate at peak load	0.1%	2.5%

Analysis: The traditional backend achieves $5\times$ higher throughput. Kubernetes API server becomes a bottleneck under high write load due to etcd contention and rate limiting (default: 1000 writes/second cluster-wide).

5.4 Resource Utilization

5.4.1 CPU Usage

Analysis: The Kubernetes controller consumes $5\times$ more CPU at idle due to:

- Continuous watch connections to API server

TABLE 5.8
CPU Consumption (millicores)

State	backend-billing	kube-billing
Idle	10m	50m
Under load (50% capacity)	100m	200m
Peak (100% capacity)	500m	800m

- Reconcile loop polling (even with no work)
- Prometheus metric collection overhead

5.4.2 Memory Usage

TABLE 5.9
Memory Consumption (MB)

State	backend-billing	kube-billing
Idle	50 MB	150 MB
Under load	150 MB	300 MB
Peak	300 MB	500 MB

Analysis: The Kubernetes controller uses $3\times$ more memory due to:

- controller-runtime cache (in-memory informer stores)
- Kubernetes client-go overhead
- Larger Go runtime footprint (more goroutines)

5.4.3 Startup Time

Analysis: The Kubernetes controller takes $5\times$ longer to start due to Kubernetes API server handshake, CRD validation, and cache synchronization.

TABLE 5.10
Startup Time (seconds)

Phase	backend-billing	kube-billing
Binary load	0.1s	0.1s
Database/API connection	0.5s	2.0s
Migrations/CRD check	1.0s	1.5s
Ready to serve	2.0s	10.0s

5.5 Operational Characteristics

5.5.1 Scaling

backend-billing:

- Manual horizontal scaling via Docker Compose replicas
- Requires external load balancer (nginx, HAProxy)
- Database connection pooling limits scale

kube-billing:

- Automatic scaling via HorizontalPodAutoscaler
- Min 1, Max 5 replicas (configurable)
- CPU target: 80%, Memory target: 80%
- Scale down stabilization: 300 seconds

Analysis: Kubernetes provides superior operational automation but adds complexity. For stable workloads, manual scaling is sufficient.

5.5.2 Fault Tolerance

backend-billing:

- Single point of failure (no built-in HA)
- PostgreSQL replication required for DB resilience
- Crash recovery: restart from last DB state

kube-billing:

- Kubernetes self-healing (automatic pod restart)
- etcd replication (3+ nodes for production)
- Crash recovery: reconcile from etcd state
- PodDisruptionBudget ensures availability during updates

Analysis: Kubernetes provides built-in fault tolerance but requires proper cluster configuration. The traditional backend requires manual HA setup.

5.5.3 Observability Coverage

TABLE 5.11
Observability Features

Feature	backend-billing	kube-billing
Custom business metrics	2	3
HTTP/Request metrics	Yes	No
Reconciliation metrics	No	Yes
Distributed tracing	Yes (HTTP + DB)	Yes (Reconcile spans)
Audit events	Manual logging	Kubernetes Events API
Status conditions	No	Yes (metav1.Condition)

Analysis: The Kubernetes system provides richer business-level observability (Events, Conditions) without custom code. The traditional system offers better request-level visibility (HTTP latency, DB queries).

5.6 Summary of Findings

Table 5.12 summarizes the comparative analysis.

TABLE 5.12
Overall Comparison (1–5 stars, higher is better)

Criterion	backend-billing	kube-billing
Code simplicity	*****	***
Development speed	*****	***
Performance (latency)	*****	**
Performance (throughput)	*****	**
Resource efficiency	*****	**
Scalability	***	*****
Fault tolerance	**	*****
Observability	****	*****
Cloud-native integration	***	*****
Portability	*****	***

5.7 Discussion

5.7.1 When to Use Kubernetes Operator Approach

The Kubernetes-based approach is advantageous when:

- **Application is already on Kubernetes** — no additional infrastructure required
- **High availability is critical** — self-healing and HPA provide resilience

- **GitOps workflow is desired** — declarative resources integrate with ArgoCD/Flux
- **Business logic is state machine-like** — reconcile pattern fits well
- **Audit trail is required** — Kubernetes Events provide built-in logging

5.7.2 When to Use Traditional Backend Approach

The traditional backend is preferable when:

- **Performance is critical** — 10–20× lower latency
- **Team lacks Kubernetes expertise** — simpler development model
- **Application must run outside Kubernetes** — portability requirement
- **Time-to-market is priority** — faster development cycle
- **Complex queries are needed** — SQL vs Kubernetes API limitations

5.7.3 Threats to Validity

Several limitations affect generalizability:

- **Local environment** — production clusters may show different performance characteristics
- **Single-node testing** — no network partition or leader election effects
- **Simulated payments** — no external payment gateway latency/failures
- **Short test duration** — no long-term etcd compaction or database bloat
- **Fixed workload** — results may vary with different access patterns

5.8 Summary

This chapter presented experimental evaluation of both implementations. The traditional backend outperforms the Kubernetes Operator in raw performance (10–20 $\times$ lower latency, 5 $\times$ higher throughput) and resource efficiency (3–5 $\times$ less CPU/memory). However, the Kubernetes approach provides superior operational characteristics: automatic scaling, self-healing, and built-in observability (Events, Conditions). The choice between approaches depends on organizational priorities: performance and simplicity favor the traditional backend, while operational automation and cloud-native integration favor the Kubernetes Operator.

Chapter 6

Conclusion

This chapter summarizes the research findings, answers the research questions posed in Chapter 1, discusses the contributions and limitations of this work, and suggests directions for future research.

6.1 Summary of Research

This thesis explored the applicability of Kubernetes as a platform for implementing business logic using Custom Resource Definitions (CRDs) and the Operator pattern. Two functionally equivalent billing systems were designed, implemented, and evaluated:

- **backend-billing:** A traditional Go backend with REST API and PostgreSQL database
- **kube-billing:** A Kubernetes Operator with CRDs and reconcile-based control loop

Both systems implement identical business logic (billing plan management,

subscription lifecycle, recurring billing, expiration handling) but differ fundamentally in architecture, state management, and execution model.

6.2 Answers to Research Questions

6.2.1 RQ1: How can business logic be effectively modeled and implemented using Kubernetes CRDs and the Operator pattern?

Answer: Business logic can be modeled using Kubernetes CRDs by:

1. **Defining domain entities as CRDs:** BillingPlan and Subscription resources extend the Kubernetes API with domain-specific semantics
2. **Using spec/status separation:** Desired state (spec) is declared by users; actual state (status) is managed by the controller
3. **Implementing reconcile loop:** The Operator continuously compares spec vs status and applies changes to converge toward desired state
4. **Leveraging Kubernetes primitives:** Finalizers for cleanup, Conditions for state reporting, Events for audit trail
5. **Using admission control:** CRD validation markers enforce business rules at API boundary

The key insight is that business logic becomes a *control problem* rather than a *computation problem*: the Operator acts as an automated control system that maintains system state according to business rules.

6.2.2 RQ2: What are the performance characteristics of CRD/Operator based implementation compared to traditional REST API architectures?

Answer: The experimental evaluation (Chapter 5) demonstrates significant performance differences:

TABLE 6.1
Performance Comparison Summary

Metric	backend-billing	kube-billing
Latency (p50, create subscription)	10 ms	100 ms
Throughput (requests/second)	~1,000	~200
CPU usage (idle)	10m	50m
Memory usage (idle)	50 MB	150 MB
Startup time	2s	10s

The traditional backend is **10–20× faster** in latency and **5× higher** in throughput. The Kubernetes API server and etcd introduce overhead from admission control, distributed consensus, and watch event propagation.

Conclusion: For performance-critical applications, the traditional approach is superior. The Operator pattern trades raw performance for operational benefits.

6.2.3 RQ3: What are the operational implications and observability capabilities of using Kubernetes as a business logic platform?

Answer: The Kubernetes approach provides significant operational advantages:

- **Automatic scaling:** HorizontalPodAutoscaler adjusts replicas based on

CPU/memory utilization

- **Self-healing:** Failed pods are automatically restarted by Kubernetes
- **Declarative deployment:** Kustomize/Helm enable version-controlled, reproducible deployments
- **Built-in observability:**
 - Kubernetes Events provide audit trail without custom logging
 - Conditions (metav1.Condition) standardize state reporting
 - Reconciliation metrics expose control loop behavior
- **GitOps compatibility:** CRD manifests integrate with ArgoCD/Flux workflows

However, these benefits come with operational complexity:

- Kubernetes cluster management required
- etcd backup and disaster recovery planning
- RBAC configuration for least-privilege access
- Monitoring of control plane health

6.2.4 RQ4: Under what circumstances is the CRD/Operator approach advantageous, and what are its limitations?

Answer: The CRD/Operator approach is advantageous when:

- **Application is already on Kubernetes** — no additional infrastructure investment
- **High availability is critical** — self-healing and HPA provide resilience
- **Business logic is state machine-like** — reconcile pattern fits naturally
- **Audit trail is required** — Kubernetes Events provide compliance logging
- **GitOps workflow is desired** — declarative resources enable PR-based changes

The approach is **not recommended** when:

- **Performance is the top priority** — 10–20× latency overhead
- **Team lacks Kubernetes expertise** — steep learning curve
- **Application must run outside Kubernetes** — portability limitation
- **Complex queries are needed** — Kubernetes API is not designed for ad-hoc queries
- **Time-to-market is critical** — slower development cycle

6.3 Contributions

This thesis makes the following contributions:

1. Two production-ready implementations:

- backend-billing: Go REST API with PostgreSQL, 86.2% test coverage, CI/CD pipeline

- kube-billing: Kubernetes Operator with CRDs, HPA, PDB, production manifests

2. **Systematic architectural comparison:**

- Code metrics (LOC, complexity, dependencies, test coverage)
- Performance benchmarks (latency, throughput, resource usage)
- Operational characteristics (scaling, fault tolerance, observability)

3. **Decision framework:** Criteria for choosing between traditional and Operator approaches based on organizational priorities

4. **Reference implementation:** Open-source codebases serving as templates for similar projects

6.4 Limitations

Several limitations affect this research:

- **Local environment:** Experiments conducted on Minikube/Docker Desktop; production clusters may show different characteristics
- **Simulated payments:** No integration with external payment gateway (Stripe, PayPal); real-world latency and failure modes not captured
- **Single workload type:** Billing domain may not represent all business logic patterns (e.g., real-time processing, batch jobs)
- **Short test duration:** No long-term effects (etcd compaction, database bloat, metric cardinality explosion)

- **Incomplete finalizer logic:** Pro-rated refund calculation not implemented in kube-billing
- **No multi-tenant testing:** Evaluation focused on single-tenant scenarios; multi-tenant isolation not assessed

6.5 Future Work

Several directions for future research emerge from this work:

6.5.1 Technical Improvements

- **Payment gateway integration:** Implement Stripe/PayPal integration for realistic payment latency and failure handling
- **Refund logic:** Complete finalizer implementation with pro-rated refund calculation
- **Multi-tenant support:** Evaluate namespace-based isolation and resource quotas
- **Webhook integration:** Add ValidationWebhook and MutationWebhook for advanced admission control
- **Backup/restore:** Implement etcd backup strategies and disaster recovery procedures

6.5.2 Research Extensions

- **Alternative domains:** Apply Operator pattern to other business domains (inventory management, order processing, user provisioning)
- **Hybrid architectures:** Explore combinations (e.g., REST API for user-facing operations, Operator for background processing)
- **Cost analysis:** Compare cloud infrastructure costs (managed Kubernetes vs managed database + compute)
- **Developer productivity:** Measure development velocity, onboarding time, and maintenance effort
- **Security analysis:** Evaluate RBAC configurations, secret management, and network policy effectiveness

6.5.3 Production Deployment

- **Helm chart:** Package kube-billing as a Helm chart for easier deployment
- **Operator Lifecycle Manager:** Publish to OperatorHub for community distribution
- **Performance tuning:** Optimize etcd configuration, reconcile concurrency, and cache settings
- **Monitoring dashboards:** Create Grafana dashboards for business metrics visualization

6.6 Final Remarks

This thesis has demonstrated that Kubernetes can serve as a platform for business logic implementation through CRDs and the Operator pattern. The approach is technically feasible and provides operational benefits (automatic scaling, self-healing, built-in observability) but comes with significant performance overhead ($10\text{--}20\times$ latency, $5\times$ lower throughput) and development complexity.

The choice between traditional and Operator-based architectures is not binary but contextual. Organizations should evaluate their specific requirements:

- If **performance, simplicity, and portability** are priorities $\rightarrow$ traditional backend
- If **operational automation, GitOps, and cloud-native integration** are priorities $\rightarrow$ Kubernetes Operator

The research contributes empirical evidence to the cloud-native architecture discourse, moving beyond anecdotal claims to data-driven decision making. Both approaches have merit; the optimal choice depends on organizational context, team expertise, and business requirements.

6.7 Publications and Artifacts

The following artifacts are available as open-source:

- **backend-billing:** <https://github.com/cuprum-acid/backend-billing>
- **kube-billing:** <https://github.com/cuprum-acid/kube-billing>
- **Thesis source:** <https://github.com/cuprum-acid/thesis>

All code is licensed under Apache 2.0 and available for community use and extension.

Bibliography cited

- [1] Kubernetes, *Production-grade container orchestration*, 2024. [Online]. Available: <https://kubernetes.io>.
- [2] M. Luksa, *Kubernetes in Action*, 1st. Manning Publications, 2017, ISBN: 9781617293726.
- [3] Cloud Native Computing Foundation, *Cloud native computing foundation*, 2024. [Online]. Available: <https://www.cncf.io>.
- [4] K. Hightower, B. Burns, and J. Beda, *Kubernetes: Up and Running: Dive into the Future of Infrastructure*, 1st. O'Reilly Media, 2017, ISBN: 978-1491935675.
- [5] J. L. Wofford, "Designing a scalable framework for declarative automation on distributed systems," 2021. arXiv: [2104.13263](https://arxiv.org/abs/2104.13263) [cs.DC]. [Online]. Available: <https://arxiv.org/abs/2104.13263>.
- [6] Kubernetes, *Custom resources*, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/>.
- [7] Kubernetes, *Operator pattern*, 2025. [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/operator/>.
- [8] J. Dobies and J. Wood, *Kubernetes Operators: Automating the Container Orchestration Platform*. O'Reilly Media, 2020, ISBN: 978-1492048046.

- [9] M. Hausenblas and S. Schimanski, *Programming Kubernetes: Developing Cloud-Native Applications*. O'Reilly Media, 2019, ISBN: 978-1492047100.
- [10] B. Ibryam and R. Huß, *Kubernetes Patterns: Reusable Elements for Designing Cloud-Native Applications*, 1st. O'Reilly Media, 2020, ISBN: 978-1492050285.
- [11] S. I. Shamim, J. A. Gibson, P. Morrison, and A. Rahman, "Benefits, challenges, and research topics: A multi-vocal literature review of kubernetes," *arXiv preprint arXiv:2211.07032*, 2022. arXiv: 2211.07032 [cs.SE]. [Online]. Available: <https://arxiv.org/abs/2211.07032>.
- [12] M. Naumovska, *Infrastructure as code in devops: Benefits, tools, and best practices*, 2025. [Online]. Available: <https://www.microtica.com/blog/what-is-infrastructure-as-code>.
- [13] Crossplane, *Crossplane is the cloud-native framework for platform engineering*, 2024. [Online]. Available: <https://www.crossplane.io>.
- [14] S. Kanthed, "Openshift operators: Automating complex application lifecycle management," *International Journal of Latest Research Perspectives*, Feb. 2024. DOI: 10.70528/IJLRP.v5.i2.1436. [Online]. Available: <https://www.ijlrp.com/research-paper.php?id=1436>.
- [15] Kubernetes SIG, *Kubebuilder: Sdk for building kubernetes apis using crds*, 2024. [Online]. Available: <https://github.com/kubernetes-sigs/kubebuilder>.
- [16] Prometheus, *Prometheus - monitoring system & time series database*, 2024. [Online]. Available: <https://prometheus.io>.
- [17] OpenTelemetry, *Opentelemetry*, 2024. [Online]. Available: <https://opentelemetry.io>.

- [18] Y. Liu, Z. Yu, Q. Wang, H. Mei, G. Song, and H. Li, “Research on cloud-native monitoring system based on Prometheus,” in *Fourth International Conference on Sensors and Information Technology (ICSI 2024)*, vol. 13107, SPIE, 2024. DOI: [10.1117/12.3029320](https://doi.org/10.1117/12.3029320). [Online]. Available: <https://doi.org/10.1117/12.3029320>.
- [19] ArgoProject, *Argo cd - declarative gitops cd for kubernetes*, 2024. [Online]. Available: <https://argo-cd.readthedocs.io>.
- [20] GitLab, *What is gitops?* 2023. [Online]. Available: <https://about.gitlab.com/topics/gitops/>.
- [21] N. Sukhija and E. Bautista, “Towards a framework for monitoring and analyzing high performance computing environments using kubernetes and prometheus,” Nov. 2019. DOI: [10.1109/SmartWorld-UIC-ATC-SCALCOM-IOP-SCI.2019.00087](https://doi.org/10.1109/SmartWorld-UIC-ATC-SCALCOM-IOP-SCI.2019.00087).
- [22] Kubernetes, *Cluster architecture*, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/architecture/>.
- [23] Kubernetes, *The kubernetes api*, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/overview/kubernetes-api/>.
- [24] Kubernetes, *Controllers*, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/architecture/controller/>.
- [25] Kubernetes, *Extending the kubernetes api*, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/>.
- [26] Kubernetes, *Extend the kubernetes api with custom resource definitions*, 2025. [Online]. Available: <https://kubernetes.io/docs/tasks/extend-kubernetes/custom-resources/custom-resource-definitions/>.

-
- [27] L. Larsson, W. Tärneberg, C. Klein, E. Elmroth, and M. Kihl, “Impact of etcd deployment on kubernetes, istio, and application performance,” 2020. arXiv: 2004.00372 [cs.NI]. [Online]. Available: <https://arxiv.org/abs/2004.00372>.
 - [28] Q. Xu, Y. Gao, and J. Wei, “An empirical study on kubernetes operator bugs,” *ACM Digital Library*, 2024. DOI: [10.1145/3650212.3680396](https://doi.org/10.1145/3650212.3680396). [Online]. Available: <https://doi.org/10.1145/3650212.3680396>.
 - [29] P. Josyula and A. Kumar, “Architectural trade-off analysis of modern service mesh designs: A qualitative atam study,” 2025. DOI: [10.1109/CICTN64563.2025.10932541](https://doi.org/10.1109/CICTN64563.2025.10932541).
 - [30] R. Chandramouli and Z. Butcher, “Building secure microservices-based applications using service-mesh architecture,” *NIST Special Publication 800-204A*, Jan. 2020. DOI: [10.6028/NIST.SP.800-204A](https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-204A.pdf). [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-204A.pdf>.